

The Algol-24 Programming Language Specification

Status: first draft complete. Nineteen chapters and five annexes, 248 rules, each verified by running the interpreter rather than by reading it.

⚠ **No rule is covered by a conformance program yet.** Every rule carries `conformance TBD`, and that number is 248 of 248. What exists is evidence — `spec/probes/` holds the programs each rule was verified with and a recording of what the implementation did — and evidence is not a commitment. Deciding which behaviours to conform *to* is a separate pass; see `spec/PLAN.md` §7.

Two silent divergences between the interpreter and the C back end are recorded in Annex C, and eighteen behaviours that look like defects in Annex D.

1. Introduction

Algol-24 is a Pascal-flavoured, gradually typed language. This document specifies its lexis, syntax, and semantics.

1.1 Authority

⚠ **The tree-walking interpreter in `compiler/*.a24` is the sole authority.** Where it and the C runtime in `bootstrap/algol.c` disagree, the interpreter defines the language and the compiler is in error. There is no other implementation and no external oracle.

⚠ **This specification is normative even where the behaviour is plainly wrong.** If the interpreter treats the Integer `0` as falsey and `0.0` as truthy, this document says so, flatly. Doubts are recorded in Annex D, which is not normative, so that the normative text never argues with itself. A specification that hedges cannot be conformed to.

Every normative statement here was verified by *running* the interpreter, not by reading it.

1.2 Conformance

An implementation conforms if, for every program, it produces the output and the exit status this document requires. Diagnostics are part of that surface: their wording is specified, because it is what a user sees and what one implementation can be compared against another by.

Where a rule is marked **compile-only** or **interpret-only**, it constrains that processor alone.

1.3 How to read a rule

Every normative statement carries an immutable identifier and its evidence:

[SRC-000] An example rule, stating one thing that a single test can prove or disprove.

```
interpreter  compiler/Scanner.a24  ScanToken
compiler     bootstrap/algol.c     alg_error
unit         Scan A Whole Program
conformance  TBD
```

⚠ **Identifiers are permanent.** They are assigned once, never reused and never renumbered; sections may be renamed and reordered freely. Numbering the sections instead would mean that inserting one clause silently rots every citation to everything after it.

The trailer is machine-readable and `spec/spec.sh` checks every line of it. The four keys do different jobs, and two of them are easy to confuse:

Key	Names	Answers
interpreter	a file and symbol in compiler/*.a24	where the authority implements this
compiler	a file and symbol in bootstrap/algol.c	where the C back end implements it, when it has a say
unit	a test in the compiler/ suite	evidence the rule <i>transcribes the authority accurately</i>
conformance	a program in conformance/	a check that <i>any implementation</i> obeys the rule

⚠ **unit is not conformance evidence, and the distinction is not pedantic.** A unit test reaches into `algol`'s own classes — it constructs a `Scanner` and asserts token types — so it tests the implementation, not the language. It is still worth citing, because [1.1] makes the interpreter normative and a test pinning the interpreter's behaviour therefore pins the language's. But another implementation has no `Scanner` class to test, so a unit test can never be run against it. Only a conformance program can.

Every rule carries a conformance line, and where no program exists yet its value is the literal `TBD` — so the gap is stated in the specification rather than left to be discovered. `spec.sh` requires the line, counts the TBDs, and `grep 'conformance TBD' ALGOL-24.md` is the corpus's backlog.

`unit` is cited where a test happens to pin the rule and omitted where none does. It is deliberately not mandatory: a unit test is a test of `algol`, and there is no intention that every rule of the language should have one. The coverage report names the rules that lack one.

Non-normative material — annexes, notes, and anything under a heading marked (*non-normative*) — carries no identifiers and constrains nothing.

2. Notation

The syntax is given in Extended Backus-Naur Form, following the Go specification's conventions:

```
Production  = production_name "=" [ Expression ] "." .
Expression  = Term { "|" Term } .
Term        = Factor { Factor } .
Factor      = production_name | token [ "..." token ] | Group | Option | Repetition .
Group       = "(" Expression ")" .
Option      = "[" Expression "]" .
Repetition  = "{" Expression "}" .
```

Lowercase production names denote lexical tokens; uppercase names denote non-terminals. Tokens are enclosed in double quotes.

The form `a ... b` means the set of characters from `a` through `b` inclusive. The symbol `∅` marks an empty alternative.

 A quoted token in this document is matched **case-insensitively if it is a keyword** and exactly otherwise. See [SRC-010].

3. Source code representation

3.1 Characters

[SRC-001] Source text is a sequence of bytes. The language attaches no encoding to it; a byte is the unit of both storage and measurement.

```
interpreter  compiler/Scanner.a24  ScanTokens
compiler     bootstrap/algol.c     alg_length
unit         Scan A Whole Program
conformance  TBD
```

[SRC-002] Outside comments, string literals and character literals, every byte must be one the scanner recognises. Any other byte is an error reading `[line N] Error: Unexpected character: C` .

```
interpreter  compiler/Scanner.a24  ScanToken
unit         Scan Unrecognized Character Is Recorded
```

conformance TBD

[SRC-003] Inside a comment, a string literal or a character literal, any byte is permitted and is carried through unchanged. A program may therefore hold text in any encoding, and the language will neither interpret nor validate it.

```
interpreter  compiler/Scanner.a24  ScanString
compiler     bootstrap/algol.c     alg_string
conformance  TBD
```

[SRC-004] Length of a String is its count of BYTES, and subscripting a String yields the byte at that position. A multi-byte character therefore has a length greater than one and can be subscripted into its parts.

```
interpreter  compiler/Interpreter.a24  LengthNative
compiler     bootstrap/algol.c         alg_length
conformance  TBD
```

Length('café') is 5, not 4, in both processors. Verified.

⚠ [SRC-002] and [SRC-003] together mean an identifier is ASCII while a string is not. The restriction is on the *program text the scanner reads*, not on the data a program may carry.

3.2 Letters and digits

[SRC-005] The following classes are used by the grammar:

```
letter      = "a" ... "z" | "A" ... "Z" | "_" | "?" .
decimal_digit = "0" ... "9" .
```

```
interpreter  compiler/Scanner.a24  IsAlpha
unit         Scan Identifier With A Question Mark
conformance  TBD
```

⚠ ? is a letter; ! is not. Gate? is a single identifier — one word to the scanner, and one word to double-click. Gate! is not: the ! is refused as an unexpected character. There is no ! operator either; the language spells negation not and inequality <> .

3.3 Line termination

[SRC-006] A line ends at #10 . It is the only byte that advances the line count used by diagnostics.

```
interpreter  compiler/Scanner.a24  ScanToken
unit        Scan Newline
unit        Scan Comment Ends At Newline
conformance TBD
```

[SRC-007] #13 is whitespace. It does not end a line, does not advance the line count, and is not required to be followed by #10 .

```
interpreter  compiler/Scanner.a24  ScanToken
conformance TBD
```

A file with CRLF endings and the same file with LF endings report identical line numbers. A lone #13 between two statements separates them as any other whitespace would. Verified.

[SRC-008] Whitespace is the space, #9 and #13 . It separates tokens and is otherwise insignificant.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance TBD
```

[SRC-009] The final line of a file need not be terminated.

```
interpreter  compiler/Scanner.a24  IsAtEnd
conformance TBD
```

3.4 Case

[SRC-010] Keywords are matched case-insensitively. `begin` , `Begin` and `BEGIN` are the same keyword.

```
interpreter  compiler/Scanner.a24  ScanIdentifier
unit        Scan Keywords
conformance TBD
```

[SRC-011] Identifiers are case-sensitive. `Xyz` and `xyz` are different names, and only the keyword *lookup* is lowered — never the lexeme a token carries.

```
interpreter  compiler/Scanner.a24  ScanIdentifier
unit        Scan Identifier
conformance TBD
```

⚠ The asymmetry is deliberate and is the one place the language departs from Pascal's uniform case-insensitivity. A program may declare `Count` and `count` as two variables; it may not declare a variable

named Begin .

4. Lexical elements

4.1 Comments

[LEX-001] A comment begins with `//` and runs to the end of the line, or to the end of the file if no `#10` follows. It is discarded and forms no token.

```
interpreter  compiler/Scanner.a24  ScanToken
unit        Scan Comment
unit        Scan Comment Ends At Newline
conformance TBD
```

[LEX-002] `///` is not a distinct form. The scanner sees `//` followed by a comment whose first character is `/` , and treats it as any other comment.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance TBD
```

The project writes documentation comments as `///` by convention, and tools may treat them specially. The language does not.

[LEX-003] There are no block comments and no nesting. `{ ... }` and `(* ... *)` are not comments: the braces are refused as unexpected characters, and the parenthesis form is read as an expression.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance TBD
```

4.2 Tokens

[LEX-004] A token is an identifier, a keyword, a literal, or an operator or item of punctuation. Whitespace and comments separate tokens and are otherwise discarded.

```
interpreter  compiler/Scanner.a24  ScanTokens
unit        Scan Tokens
conformance TBD
```

[LEX-005] Where a shorter and a longer token both match, the longer is taken. `<` followed by `>` is one `<>` ; `<` followed by anything else is a `<` on its own.

```
interpreter  compiler/Scanner.a24  ScanToken
unit         Scan Less Is Not Greedy
conformance  TBD
```

`<<><=<` scans as `<`, `<>`, `<=`, `<` — four tokens.

[LEX-006] There is no automatic semicolon insertion. A line ending is whitespace and never stands in for a `;`.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance  TBD
```

4.3 Identifiers

[LEX-007] An identifier is a letter followed by any number of letters and digits. `letter` and `decimal_digit` are as defined in [SRC-005], so `_` and `?` are letters.

```
identifier = letter { letter | decimal_digit } .
```

```
interpreter  compiler/Scanner.a24  ScanIdentifier
unit         Scan Identifier
unit         Scan Identifier With A Question Mark
conformance  TBD
```

[LEX-008] Because `?` and `_` are letters, either may begin an identifier, and `?` alone is a well-formed identifier.

```
interpreter  compiler/Scanner.a24  IsAlpha
conformance  TBD
```

`var ?abc := 7;` declares a variable. So does `var ? := 7;`. See Annex D.

[LEX-009] An identifier may not be spelled the same as a keyword in any case, because the keyword is recognised first. `var begin := 7;` and `var BEGIN := 7;` are both refused with `Expect variable name`.

```
interpreter  compiler/Scanner.a24  ScanIdentifier
unit         Scan Keywords
conformance  TBD
```

4.4 Keywords

[LEX-010] The following 38 words are keywords and are matched case-insensitively per [SRC-010]:

and	as	begin	break	case	class	const	constructor
do	else	end	except	exit	false	for	function
if	in	is	nil	not	object	of	or
print	private	procedure		public	raise	super	then
this	true	try	type	uses	var	while	

```
interpreter  compiler/Scanner.a24  Keywords
unit        Scan Keywords
conformance  TBD
```

[LEX-011] `unit`, `test` and `on` are **not** keywords. They are ordinary identifiers that the grammar recognises by position — `unit` opening a file, `test` before a block's quoted name, `on` introducing a handler — and each may be used as a variable name.

```
interpreter  compiler/Scanner.a24  Keywords
interpreter  compiler/Parser.a24   UnitHeader
unit        Parse On Is Not A Keyword
conformance  TBD
```

`var test := 7;` is a valid declaration, and so are the `unit` and `on` forms. Verified in all three.

4.5 Operators and punctuation

[LEX-012] The following are operators and punctuation:

(	)	[	]	,	.	;	:
+	-	*	/	=	:=		
<	<=	>	>=	<>			

```
interpreter  compiler/Scanner.a24  ScanToken
unit        Scan Operators
conformance  TBD
```

[LEX-013] `=` is equality and `:=` is assignment. Inequality is `<>`. There is no `==`, no `!` and no `!=`; `!` is not a character the scanner accepts anywhere outside a comment or a literal.

```
interpreter  compiler/Scanner.a24  ScanToken
interpreter  compiler/TokenType.a24 TOKEN_ASSIGN
```



```
unit          Scan Operators
conformance   TBD
```

[LEX-014] `and` , `or` , `not` , `in` , `is` and `as` are operators spelled as keywords rather than punctuation, and are subject to [SRC-010].

```
interpreter    compiler/Scanner.a24  Keywords
unit          Scan Keywords
conformance    TBD
```

⚠ `{` and `}` are not tokens of the language at all — not as comment delimiters, not as block delimiters, and not as set constructors. A block is `begin ... end` ; a collection literal uses `[ and ]` .

4.6 Integer literals

[LEX-015] An integer literal is a run of decimal digits.

```
integer_lit = decimal_digit { decimal_digit } .
```

```
interpreter    compiler/Scanner.a24  ScanNumber
unit          Scan Number
unit          Scan Integer Is Not A Double
conformance    TBD
```

[LEX-016] Decimal is the only base. There is no hexadecimal, octal or binary form, and no digit separator.

`0x10` is not a number: it scans as the integer `0` followed by the identifier `x10` .

```
interpreter    compiler/Scanner.a24  ScanNumber
conformance    TBD
```

[LEX-017] Leading zeros are permitted and carry no meaning. `007` is the integer 7, not an octal.

```
interpreter    compiler/Scanner.a24  ScanNumber
conformance    TBD
```

[LEX-018] An Integer is a signed 32-bit value. A literal outside that range **wraps silently**, with no diagnostic:

`2147483648` is the integer `-2147483648` , and `999999999999999` is `276447231` . Arithmetic wraps the same way, so `2147483647 + 1` is `-2147483648` .

```
interpreter    compiler/Scanner.a24  ToInteger
compiler       bootstrap/algol.c     alg_add
conformance    TBD
```

Both processors produce identical wrapped values. See Annex D.

[LEX-019] There is no negative literal. A leading `-` is the unary operator applied to a non-negative literal, which is why `2-1` is a subtraction rather than two adjacent expressions.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance  TBD
```

4.7 Double literals

[LEX-020] A double literal requires at least one digit on **both** sides of the point.

```
double_lit = decimal_digit { decimal_digit } "." decimal_digit { decimal_digit } .
```

```
interpreter  compiler/Scanner.a24  ScanNumber
unit         Scan Number Decimal
conformance  TBD
```

[LEX-021] `1.` is therefore not a double. It is the integer `1` followed by the `.` operator, and a program containing it fails with `Expect property name after '.'`. Likewise `.5` is not a literal at all.

```
interpreter  compiler/Scanner.a24  ScanNumber
unit         Scan Integer Then Dot
conformance  TBD
```

[LEX-022] There is no exponent notation. `1e5` scans as the integer `1` followed by the identifier `e5`.

```
interpreter  compiler/Scanner.a24  ScanNumber
conformance  TBD
```

4.8 Character literals

[LEX-023] A quoted literal enclosing exactly one byte **of source** is a `Char` rather than a `String`. The measurement is taken on the raw span between the quotes, not on the value.

```
interpreter  compiler/Scanner.a24  ScanString
unit         Scan One Character Is A Char
conformance  TBD
```

[LEX-024] A Char may also be written `#` followed by decimal digits, giving the character with that code point: `#65` is `A` and `#10` is a line feed. A `#` not followed by a digit is an error reading `[line N] Error: Invalid character: C`.

```
char_lit = "" source_byte "" | "#" decimal_digit { decimal_digit } .
```

```
interpreter  compiler/Scanner.a24  ScanChar
unit         Scan Char By Code Point
unit         Scan Char Without Digits
conformance  TBD
```

[LEX-025] A Char's code point is limited to 0 ... 127. A larger value is a runtime error reading `Char` is limited to `0..127`.

```
interpreter  compiler/Interpreter.a24  CharNative
compiler     bootstrap/algol.c         alg_char
conformance  TBD
```

[LEX-026] A Char and a String are never equal, however alike they look. `'a' = 'a'` is true because both sides are Chars; `Copy('abc', 0, 1) = 'a'` is **false**, because `Copy` yields a String of length one and the Char `'a'` is not it.

```
interpreter  compiler/Interpreter.a24  IsEqual
compiler     bootstrap/algol.c         equals
conformance  TBD
```

4.9 String literals

[LEX-027] A string literal is enclosed in single quotes. A quote within it is written twice.

```
string_lit = "" { source_byte_other_than_quote | "'" } "" .
```

```
interpreter  compiler/Scanner.a24  ScanString
unit         Scan String
unit         Scan Doubled Quote
conformance  TBD
```

[LEX-028] There are no backslash escapes. `'a\nb'` is four bytes, and its element at index 1 is the backslash itself. A line feed is written `#10` and concatenated.

```
interpreter  compiler/Scanner.a24  ScanString
conformance  TBD
```

[LEX-029] `''` is the empty String. `''''` is a String of length one holding a quote — **not** a Char, because [LEX-023] measures the source span, which is two bytes.

```
interpreter  compiler/Scanner.a24  ScanString
unit         Scan Empty String
unit         Scan An Escaped Quote Is A String
conformance  TBD
```

[LEX-030] A string literal may span lines. The line feed is part of its value and advances the line count, so `'one ↵ two'` is seven bytes.

```
interpreter  compiler/Scanner.a24  ScanString
conformance  TBD
```

[LEX-031] A string that reaches the end of the file unclosed is an error reading [line N] Error: Unterminated string. , where N is the line the scan reached — not the line the string opened on.

```
interpreter  compiler/Scanner.a24  ScanString
unit         Scan Unterminated String
conformance  TBD
```

[LEX-032] A String cannot hold the Char whose code point is 0. The Char itself is well formed and `#0` is Char is true, but concatenating it into a String truncates the String there: `Length('a' + Str(#0) + 'b')` is 2.

```
interpreter  compiler/Interpreter.a24  StrNative
compiler     bootstrap/algol.c         alg_string
conformance  TBD
```

Both processors agree. See Annex D.

5. Constants and variables

5.1 Variable declarations

[VAR-001] A variable is introduced by `var` , with an optional declared type and an optional initializer.

```
VarDecl = "var" identifier [ ":" Type ] [ "!=" Expression ] ";" .  
Type    = identifier [ "of" identifier ] .
```

```
interpreter  compiler/Parser.a24  VarDeclaration  
unit        Parse Var Statement  
unit        Parse Var Expect Semicolon  
conformance TBD
```

[VAR-002] A variable declared without an initializer holds `nil`, whatever its declared type. **There is no zero value:** an uninitialized `Integer` is `nil`, not `0`.

```
interpreter  compiler/Interpreter.a24  VisitVarStmt  
conformance TBD
```

[VAR-003] A declared type constrains the initializer and every later assignment. A violation is the error `Type mismatch!`

```
interpreter  compiler/TypeChecker.a24  Assignable  
conformance TBD
```

[VAR-004] There is **no implicit numeric conversion** in a declaration. `var X : Integer := 1.5;` is a mismatch, and so is `var X : Double := 1;` — widening is refused as firmly as narrowing.

```
interpreter  compiler/TypeChecker.a24  Assignable  
conformance TBD
```

See Annex D.

[VAR-005] `nil` satisfies every declared type, so `var X : Integer := nil;` is accepted.

```
interpreter  compiler/TypeChecker.a24  Assignable  
compiler     bootstrap/algol.c         alg_is  
conformance TBD
```

[VAR-006] Any is the declared type meaning *not known*. It accepts every value and is compatible in both directions.

```
interpreter  compiler/TypeChecker.a24  Assignable  
conformance TBD
```

[VAR-007] A name may not be declared twice in one scope. The second is refused with 'X' is already defined.

```
interpreter  compiler/Resolver.a24  CheckDuplicates
unit        Resolve Duplicate Variable
conformance  TBD
```

[VAR-008] A collection may carry an element type, written of : var L : List of Integer := []; . The element type is honoured only after List .

```
interpreter  compiler/Parser.a24  VarDeclaration
conformance  TBD
```

5.2 Sections

[VAR-009] In the header of a function, procedure, class or object, var may open a **section**: a run of declarations, each ending in ; , closed by the next section marker or by begin .

```
VarSection = "var" { identifier { "," identifier } [ ":" Type ] [ "!=" Expression ] ";" } .
```

```
interpreter  compiler/Parser.a24  DeclarationSection
unit        Parse A Function Local Var Section
conformance  TBD
```

[VAR-010] Within a section several names may share one declaration: A, B : Integer; declares both.

```
interpreter  compiler/Parser.a24  DeclarationSection
unit        Parse A Comma Group Stays A Group
unit        Parse A Single Name Is Not A Group
conformance  TBD
```

[VAR-011] A section is a feature of a **header**, not of a program body. At the top level var declares exactly one name, and a run of declarations beneath it is read as ordinary statements — var A : Integer; followed by B : String; fails on the second with Expect ';' after expression.

```
interpreter  compiler/Parser.a24  ReadDeclarationSections
conformance  TBD
```

5.3 Constants

[VAR-012] A constant is introduced by `const` and **must** be given a value. Omitting the initializer is refused with A constant must be given a value.

```
ConstDecl = "const" identifier [ ":" Type ] "!=" Expression ";" .
```

```
interpreter  compiler/Parser.a24  ConstDeclaration
unit        Parse A Constant Must Be Given A Value
conformance  TBD
```

[VAR-013] A constant may not be assigned to. The attempt is refused with Can't assign to constant 'C'.

```
interpreter  compiler/Resolver.a24  VisitAssignExpr
conformance  TBD
```

[VAR-014] ⚠ A constant's initializer is an **ordinary expression evaluated at run time**, not a constant expression. `const C := 1 + 2;` is legal, and so is `const C := V;` where `V` is a variable — the constant takes whatever value `V` held at that moment.

`const` therefore means *this binding may not be reassigned*, not *this value is known before the program runs*.

```
interpreter  compiler/Interpreter.a24  VisitVarStmt
conformance  TBD
```

[VAR-015] `const` may open a section on the same terms as `var`, and the two may appear together in one header.

```
interpreter  compiler/Parser.a24  DeclarationSection
unit        Parse A Const Section
unit        Parse Var And Const Sections Together
conformance  TBD
```

6. Types

6.1 The kinds of type

[TYP-001] A value has exactly one of these runtime types:

Kind	Types
------	-------

Kind	Types
Primitive	Integer , Double , String , Char , Boolean
Declared	a class type, an enumeration type
Collection	List , Set , Stack , Array , Map
Resource	Buffer , TextFile
Unknown	Any — a declaration only, never a runtime type

```

interpreter  compiler/ObjFunction.a24  TypeNameOf
compiler     bootstrap/algol.c        alg_is
conformance  TBD

```

[TYP-002] A type is written as an identifier. The only compound form is `List of T` , which names an element type.

```
Type = identifier [ "of" identifier ] .
```

```

interpreter  compiler/Parser.a24  VarDeclaration
conformance  TBD

```

[TYP-003] `Char` and `String` are distinct types, and a one-character value is a `Char` . `'s'` is `String` is **false**.

```

interpreter  compiler/ObjFunction.a24  TypeNameOf
conformance  TBD

```

6.2 Any

[TYP-004] `Any` is the declared type meaning *the type is not known*. It is compatible with every type in both directions, and no value ever reports `Any` as its runtime type.

```

interpreter  compiler/TypeChecker.a24  Assignable
conformance  TBD

```

6.3 nil

[TYP-005] `nil` is of no type at all. `nil is T` is **false** for every `T` , including the type `nil` was declared as.


```
interpreter  compiler/Interpreter.a24  VisitIsExpr
compiler     bootstrap/algol.c         alg_is
conformance  TBD
```

[TYP-006] `nil` nonetheless satisfies every declared type for the purpose of assignment — see [VAR-005]. A value that is not there has no type to check, and is accepted everywhere.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  TBD
```

6.4 Collection types

[TYP-007] The five collection types are distinguished by their kind, and each answers `is` to its own name only. A `List` is not a `Set`.

```
interpreter  compiler/ObjCollection.a24  Kind
conformance  TBD
```

[TYP-008] `Array` is fixed in size. Its elements begin as `nil`, it is indexed from zero, and an index outside its bounds is the runtime error `Index N out of range 0..M`. It does not grow on assignment.

```
interpreter  compiler/ObjCollection.a24  At
compiler     bootstrap/algol.c           alg_subscript_set
conformance  TBD
```

[TYP-009] A collection is **not** a class instance. It has no `ClassName`, and asking for one is the error `Undefined property 'ClassName'`.

```
interpreter  compiler/ObjCollection.a24  Get
conformance  TBD
```

6.5 What a class type cannot do

These rules are normative in their own right, and together they say which built-in behaviour a program cannot reproduce for a type of its own. Annex E takes up what it would cost to lift each one.

[TYP-010] A class instance may not be subscripted. `B[0]` on an instance is the runtime error `Subscript target should be an ordinal`, whatever methods the class declares.

```
interpreter  compiler/Interpreter.a24  VisitSubscriptExpr
conformance  TBD
```

[TYP-011] A class instance may not be iterated. `for var X in B do` over an instance is the runtime error
Can only iterate a collection or a String.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
conformance  TBD
```

[TYP-012] A class exposes a **field** without parentheses and a **method** with them. There is no getter declaration, so a computed value cannot be read as a property: a method named `Length` read as `B.Length` yields the function itself, printing `<fn Length>`, where a collection's `Length` yields its count.

```
interpreter  compiler/ObjInstance.a24  Get
conformance  TBD
```

7. Properties of types and values

7.1 Assignability

[VAL-001] A value is assignable to a declaration when its type is the declared type, when either is `Any`, when the value is `nil`, or when its class inherits from the declared class.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  TBD
```

[VAL-002] Nothing else converts. There is no numeric widening — see [VAR-004] — and no conversion between `Char` and `String`.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  TBD
```

7.2 Type tests

[VAL-003] `X is T` is true when the runtime type of `X` is `T`, or is a class inheriting from `T`. Inheritance is followed upward only: a `Dog` is an `Animal`, and an `Animal` is not a `Dog`.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
compiler     bootstrap/algol.c         alg_is
conformance  TBD
```

[VAL-004] `nil is T` is **false** for every `T`. A value that is not there has no type to test.

```

interpreter  compiler/Interpreter.a24  VisitIsExpr
compiler     bootstrap/algol.c        alg_is
conformance  TBD

```

[VAL-005] `X is Any` is **false** for every `X`. `Any` is a declaration, never a runtime type.

```

interpreter  compiler/Interpreter.a24  VisitIsExpr
conformance  TBD

```

[VAL-006] The type name in `is` is matched case-insensitively, so `1 is integer` is true.

```

interpreter  compiler/Interpreter.a24  VisitIsExpr
conformance  TBD

```

7.3 Casts

[VAL-007] `X as T` has **no runtime effect whatever**. It tells the checker what the programmer claims, and nothing verifies the claim: `X as Integer` where `X` holds `'text'` yields `'text'` and raises nothing.

```

interpreter  compiler/TypeChecker.a24  Reduce
conformance  TBD

```

See Annex D.

7.4 Truthiness

[VAL-008] A value is **falsey** if it is `nil`, `False`, the Integer `0`, or an enumeration member whose ordinal is `0`. Every other value is **truthy**.

⚠ In particular these are all **truthy**: `0.0`, the empty String `''`, the empty List `[]`, the empty Map `[:]`, and every `Char`.

```

interpreter  compiler/Interpreter.a24  IsTruthy
compiler     bootstrap/algol.c        alg_truthy
unit         Execute Logical Truthy
unit         Evaluate Unary Bang Nil
conformance  TBD

```

See Annex D.

7.5 Equality

[VAL-009] `=` and `<>` promote numerically. `1 = 1.0` is true, and so is `0 = 0.0`.

```
interpreter  compiler/Interpreter.a24  IsEqual
compiler    bootstrap/algol.c         equals
unit        Evaluate Binary Equal Equal
conformance TBD
```

[VAL-010] A `Char` is never equal to a `String` — see [LEX-026].

```
interpreter  compiler/Interpreter.a24  IsEqual
conformance TBD
```

[VAL-011] Class instances, collections and enumeration members compare by **identity**, not by contents. `[1, 2] = [1, 2]` is **false**: they are two collections. Two references to one collection are equal.

```
interpreter  compiler/Interpreter.a24  IsEqual
compiler    bootstrap/algol.c         equals
conformance TBD
```

[VAL-012] `nil = nil` is true.

```
interpreter  compiler/Interpreter.a24  IsEqual
unit        Evaluate Binary Bang Equal Nil
conformance TBD
```

7.6 Membership

[VAL-013] ⚠ `in`, `Contains`, and `Map` key lookup compare **strictly**, with no numeric promotion. `1 in [1.0]` is **false**, `1.0 in [1]` is **false**, and a `Map` holding the key `1` does not contain the key `1.0` — while `1 = 1.0` is true.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler    bootstrap/algol.c           strict_equals
conformance TBD
```

Equality and membership therefore disagree, deliberately in each case and incompatibly with each other. See Annex D.

7.7 Ordering

[VAL-014] `<`, `<=`, `>` and `>=` apply to numbers and to `Char` only. **Strings are not ordered:** `'ab' < 'cd'` is the runtime error `Operands must be numbers`.

```
interpreter  compiler/Interpreter.a24  VisitBinary
unit        Evaluate Binary Greater Left Not Number
conformance TBD
```

A program needing to order text must compare it character by character, which is what `compiler/CEmitter.a24`'s `TextLess` does.

8. Declarations and scope

8.1 Blocks

[DCL-001] A block is `begin ... end` and introduces a scope. A name declared inside it is not visible after it: reading one is `Undefined variable 'X'`.

```
interpreter  compiler/Resolver.a24  BeginScope
conformance TBD
```

[DCL-002] A block sees every name of the scopes enclosing it.

```
interpreter  compiler/Resolver.a24  ResolveLocal
unit        Resolve One Hop
unit        Resolve Two Hops
conformance TBD
```

8.2 Shadowing

[DCL-003] A declaration shadows an outer one of the same name for the rest of its scope. The outer binding is untouched and reappears when the scope ends.

```
interpreter  compiler/Resolver.a24  Declare
unit        Resolve Same Level
conformance TBD
```

[DCL-004] A `var` may shadow a `const`. The inner name is an ordinary variable and may be assigned; the outer constant is unaffected.

```
interpreter  compiler/Resolver.a24  DeclareBinding
conformance  TBD
```

8.3 Declaration and use

[DCL-005] A local may not be read in its own initializer. `var X := X;` inside a block is refused with `Can't read local variable in its own initializer.`, even where an outer `X` exists.

```
interpreter  compiler/Resolver.a24  ResolveLocal
unit         Resolve Local Variable Is Own Initializer
conformance  TBD
```

[DCL-006] Declarations take effect **in order of execution**. A top-level name does not exist until its declaration has run, so calling a function written below the call fails with `Undefined variable 'F'`.

```
interpreter  compiler/Interpreter.a24  VisitFunctionStmt
conformance  TBD
```

[DCL-007] A free name in a function body is resolved **when the body runs**, not where it is written. Two functions may therefore call each other, provided neither is called before both declarations have run.

```
interpreter  compiler/Interpreter.a24  LookupVariable
conformance  TBD
```

[DCL-006] and [DCL-007] together are why mutual recursion works while a forward call does not.

8.4 Loop variables

[DCL-008] A variable declared in a `for` header belongs to the loop, in both forms, and is not visible after it ends.

For the counted form this follows from [DCL-001] rather than being a rule of its own: `for` **desugars into a block** holding the initializer and a `while`, so the variable is scoped because it is inside a block.

```
interpreter  compiler/Parser.a24      ForStatement
interpreter  compiler/Resolver.a24    VisitForInStmt
unit         Parse For Statement
conformance  TBD
```

8.5 this and super

[DCL-009] this outside a class is refused with Can't use 'this' outside a class.

```
interpreter  compiler/Resolver.a24  VisitThisExpr
unit        This Is Never Caught
conformance TBD
```

[DCL-010] super outside a class is refused with Can't use 'super' outside a class. , and inside a class having no superclass with Can't use 'super' in a class with no superclass.

```
interpreter  compiler/Resolver.a24  VisitSuperExpr
conformance TBD
```

8.6 Visibility

[DCL-011] private: and public: are section markers within a class or object, each governing the members that follow it. A member declared under no marker is public.

```
interpreter  compiler/Parser.a24  ReadDeclarationSections
unit        A Public Member Is Reachable From Outside
unit        A Private Field Is Not Readable From Outside
conformance TBD
```

[DCL-012] The body starts public however the header ended. A private: in the header does not carry across begin .

```
interpreter  compiler/Parser.a24  ReadDeclarationSections
unit        The Body Starts Public However The Header Ended
conformance TBD
```

[DCL-013] Privacy belongs to the **class**, not to the object. A method may reach the private members of another instance of its own class.

```
interpreter  compiler/TypeChecker.a24  CheckVisibility
unit        Another Instance Of The Same Class Reaches Its Privates
conformance TBD
```

[DCL-014] A subclass does not reach what its parent hid. Reading a parent's private member through a receiver declared as the parent is refused with 'N' is private to P.

```
interpreter  compiler/TypeChecker.a24  CheckVisibility
unit        A Subclass Does Not Reach What Its Parent Hid
conformance TBD
```

[DCL-015] ⚠ Visibility is checked **statically, and only where the receiver's type is known**. Reached through a bare name inside a method — which resolves through `this` — or through a receiver declared Any, a private member is readable and writable from anywhere.

interpreter compiler/TypeChecker.a24 CheckVisibility
unit A Private Member Is Caught Through A Declared Receiver
conformance TBD

var C : Any := Counter(); C.Count yields the private field. A subclass reading its parent's private member by bare name gets it, while the same member through a typed receiver is refused. See Annex D.

9. Expressions

9.1 Precedence and associativity

[EXP-001] Operators bind in this order, tightest first:

	Operators
1	f(...) call · a[i] subscript · a.b property
2	- unary · not
3	* · /
4	+ · -
5	< · <= · > · >= · in · is
6	= · <>
7	and
8	or
9	as
10	:=

interpreter compiler/Parser.a24 Expression
unit Parse Term Plus
unit Parse Factor Star
conformance TBD

$1 + 2 * 3$ is 7, $-2 * 3$ is -6, not True and False is false, True or False and False is true, and $False = False$ and False is false. Each distinguishes its pair. Verified.

[EXP-002] Binary operators of one level are left-associative: $10 - 2 - 3$ is 5 and $12 / 2 / 3$ is 2.

interpreter compiler/Parser.a24 Term
conformance TBD

[EXP-003] `as` binds looser than `or`, so `A and B as C` casts the whole conjunction rather than `B`. Because a cast has no runtime effect [VAL-007], this is observable only through the checker.

interpreter compiler/Parser.a24 Expression
conformance TBD

9.2 Arithmetic

[EXP-004] Integer arithmetic yields an Integer, **including** `/`, which divides and truncates toward zero: $7 / 2$ is 3 and $-7 / 2$ is -3.

interpreter compiler/Interpreter.a24 VisitBinary
compiler bootstrap/algol.c alg_divide
unit Evaluate Binary Slash
conformance TBD

[EXP-005] A Double on either side promotes the operation and the result: $7.0 / 2$ and $7 / 2.0$ are both 3.5, and $1 + 2.0$ is 3.0.

interpreter compiler/Interpreter.a24 VisitBinary
unit Evaluate Binary Plus Mixed
conformance TBD

[EXP-006] ⚠ Integer division by zero is the runtime error `Division by zero`. **Double division by zero is not an error:** it yields `Infinity`, `-Infinity` or `NaN`, and the program continues.

interpreter compiler/Interpreter.a24 VisitBinary
compiler bootstrap/algol.c alg_divide
conformance TBD

See Annex D.

[EXP-007] Arithmetic wraps silently at the bounds of a 32-bit Integer — see [LEX-018].

```
compiler    bootstrap/algol.c  alg_add
conformance TBD
```

9.3 Concatenation

[EXP-008] `+` concatenates when **either** operand is a String or a Char, converting the other. `'x' + 1` is `x1`, `1 + 'x'` is `1x`, and `'a' + 'b'` — two Chars — is the String `ab`.

```
interpreter compiler/Interpreter.a24 VisitBinary
compiler    bootstrap/algol.c      alg_add
unit        Evaluate Binary Plus String
conformance TBD
```

9.4 Logical operators

[EXP-009] `and` and `or` **short-circuit**. The right operand is evaluated only when the left does not decide the result.

```
interpreter compiler/Interpreter.a24 VisitLogical
unit        Execute Logical And
unit        Execute Logical Or
conformance TBD
```

[EXP-010] Both operators test truthiness **[VAL-008]** rather than requiring a Boolean.

```
interpreter compiler/Interpreter.a24 IsTruthy
unit        Execute Logical Truthy
conformance TBD
```

9.5 Calls

[EXP-011] A call checks arity. A mismatch is `Expected N arguments but got M`.

```
interpreter compiler/Interpreter.a24 VisitCall
unit        Call Wrong Number Of Arguments
conformance TBD
```

[EXP-012] Calling something that is neither a function nor a class is `Can only call functions and classes`.

```
interpreter compiler/Interpreter.a24 VisitCall
unit        Call Non Function
```

conformance TBD

[EXP-013] Where a name is overloaded, selection is made on the **whole signature** — the number of arguments and the type of each — and is made **at run time**, from the arguments actually passed.

```
interpreter  compiler/ObjClass.a24  FindOverload
compiler     bootstrap/algol.c      alg_invoke
conformance  TBD
```

[EXP-014] When no overload fits, the call fails with No matching signature for function.

⚠ A Char does not fit a String parameter [LEX-026], so M('x') selects no overload declared String — a one-character argument is not the type it looks like.

```
interpreter  compiler/ObjClass.a24  FindOverload
conformance  TBD
```

9.6 Subscripting

[EXP-015] Subscripting a String yields the Char at that byte position, counted from zero. An index outside the value is Index N out of range 0..M.

```
interpreter  compiler/Interpreter.a24  VisitSubscript
compiler     bootstrap/algol.c         alg_subscript_get
conformance  TBD
```

[EXP-016] A class instance may not be subscripted — see [TYP-010].

```
interpreter  compiler/Interpreter.a24  VisitSubscript
conformance  TBD
```

9.7 Assignment

[EXP-017] Assignment is an **expression**, and its value is the value assigned: X := (Y := 1) leaves both at 1.

```
interpreter  compiler/Parser.a24  Assignment
unit         Resolve Assignment
conformance  TBD
```

10. Statements

10.1 Blocks and expression statements

[STM-001] A block is `begin ... end` and may be empty.

```
interpreter  compiler/Interpreter.a24  VisitBlockStmt
unit        Execute Block Statement
conformance  TBD
```

[STM-002] A declaration is a statement, so a `var` may stand as the body of a branch or a loop without a block around it.

```
interpreter  compiler/Parser.a24  Statement
conformance  TBD
```

10.2 Conditionals

[STM-003] `if Cond then S` with an optional `else S`. The condition is tested for truthiness [VAL-008], not required to be a Boolean. A missing `then` is Expect 'then' after if condition.

```
interpreter  compiler/Interpreter.a24  VisitIfStmt
unit        Execute If Statement
unit        Execute Else Statement
unit        Parse If Expect Then
conformance  TBD
```

[STM-004] An `else` binds to the **nearest** unmatched `if`.

```
interpreter  compiler/Parser.a24  IfStatement
conformance  TBD
```

10.3 Loops

[STM-005] `while Cond do S`. A missing `do` is Expect 'do' after condition.

```
interpreter  compiler/Interpreter.a24  VisitWhileStmt
unit        Execute While Loop
unit        Parse While Expect Do
conformance  TBD
```

[STM-006] The counted form is `for Init ; Cond ; Step do S`, and it **desugars into a block** holding the initializer and a `while` — which is why its variable is scoped [DCL-008].

```
interpreter  compiler/Parser.a24  ForStatement
unit         Execute For Loop
unit         Parse For Statement
conformance  TBD
```

[STM-007] `for var X in C do S` walks a collection or a String. Over a String it yields each `Char`; over a Map it yields each **key**.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
conformance  TBD
```

[STM-008] Iterating anything else is `Can only iterate a collection or a String.` — see [TYP-011].

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
conformance  TBD
```

[STM-009] ⚠ The collection is **snapshotted** when the loop begins. Adding to it inside the loop does not lengthen the walk.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
compiler     bootstrap/algol.c         alg_iterable
conformance  TBD
```

[STM-010] `break` leaves the innermost enclosing loop. Outside a loop it is refused **where it is written** — a parse-time check — with `Must be inside a loop to use 'break'.`

```
interpreter  compiler/Parser.a24  BreakStatement
unit         Parse Break Inside A While
unit         Parse Break Outside A Loop
conformance  TBD
```

10.4 Case

[STM-011] A `case` names a value, then arms, then `end`. An arm may list several values separated by commas, and a final `else` is optional.

```
CaseStmt = "case" Expression "of" { Arm } [ "else" Statement ] "end" .
Arm       = Expression { "," Expression } ":" Statement .
```

```
interpreter  compiler/Parser.a24  CaseStatement
conformance  TBD
```

[STM-012] ⚠ `case` **desugars into an if/else-if chain**. There is no case statement downstream of the parser, and two consequences follow from that rather than from any rule of their own:

- Arms compare with `=` [VAL-009], so a `Double` matches an `Integer` arm of the same value — `1.0` matches the arm `1`.
- There is no fall-through. At most one arm runs.

```
interpreter compiler/Parser.a24 CaseStatement conformance TBD
```

[STM-013] When no arm matches and there is no `else`, nothing happens and execution continues after the `end`.

```
interpreter  compiler/Parser.a24  CaseStatement
conformance  TBD
```

10.5 Exit

[STM-014] `Exit` returns from the enclosing function or procedure, with a value in a function and bare in a procedure. Statements after it do not run.

```
interpreter  compiler/Interpreter.a24  VisitReturnStmt
conformance  TBD
```

[STM-015] `Exit` at the top level is refused with `Can't return from top-level code`.

```
interpreter  compiler/Resolver.a24  VisitReturnStmt
unit          Invalid Return
conformance  TBD
```

10.6 Exceptions

[STM-016] `raise E` carries **any value** — a `String`, an `Integer`, a class instance, anything.

```
interpreter  compiler/Interpreter.a24  VisitRaiseStmt
unit          Parse Raise
conformance  TBD
```

[STM-017] A handler is written `on e : T do S` and matches on the runtime type name of the raised value. A handler for a base class catches a derived value.

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
unit         Parse Try With A Typed Handler
conformance  TBD
```

[STM-018] ⚠ The **most derived** matching handler runs, however the handlers are ordered. Writing the base first does not shadow the derived one.

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
compiler     bootstrap/algol.c         alg_handler
conformance  TBD
```

[STM-019] A handler written without `on` is the catch-all and matches any raised value.

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
unit         Parse Untyped Handler Is The Catch All
unit         Parse Empty Except Is The Catch All
conformance  TBD
```

[STM-020] A runtime error raised by the language is catchable **as a String**, carrying the diagnostic as its value: dividing by zero inside a `try` is caught by `on e : String with e equal to Division by zero.`

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
compiler     bootstrap/algol.c         alg_error
conformance  TBD
```

[STM-021] A value raised and never caught ends the program, printing `Uncaught:` followed by the value, and exits with status **70**.

```
interpreter  compiler/Main.a24  Main
compiler     bootstrap/algol.c  alg_raise
conformance  TBD
```

10.7 print

[STM-022] `print E` writes the stringified value followed by a newline.

```
interpreter  compiler/Interpreter.a24  VisitPrintStmt
unit         Execute Print Statement
conformance  TBD
```

11. Functions, procedures and closures

11.1 Declarations

[FUN-001] A subprogram is declared `function` or `procedure`. Parameters may be typed or untyped, and a return type is optional.

```
FunDecl = ( "function" | "procedure" ) identifier "(" [ Params ] ")"
          [ ":" Type ] ";" [ Sections ] Block .
Params  = identifier [ ":" Type ] { "," identifier [ ":" Type ] } .
```

```
interpreter  compiler/Parser.a24  ParseFunction
unit         Parse Function
unit         Parse Function No Open Parenthesis
conformance  TBD
```

[FUN-002] A subprogram that returns without a value yields `nil`.

```
interpreter  compiler/Interpreter.a24  VisitReturnStmt
conformance  TBD
```

[FUN-003]  `function` and `procedure` are **not distinguished** by the language. A procedure may `Exit` a value, and a caller may use the result of either. The choice of word documents intent and constrains nothing.

```
interpreter  compiler/Parser.a24  ParseFunction
conformance  TBD
```

[FUN-004] A declaration may not have more than 255 parameters: Can't have more than 255 parameters.

```
interpreter  compiler/Parser.a24  ParseFunction
unit         Parse Function More Than 255 Parameters
conformance  TBD
```

11.2 Parameters and results

[FUN-005] A call checks the number of arguments — see [EXP-011].

```
interpreter  compiler/Interpreter.a24  VisitCall
conformance  TBD
```


[FUN-006] ⚠ A **top-level** subprogram's declared parameter types are **not enforced**. function `F(N : Integer)` accepts a `String`, a `Double` or a `Boolean` without complaint, from a literal or through a variable of any declared type.

```
interpreter  compiler/Interpreter.a24  VisitCall
conformance  TBD
```

[FUN-007] A **method's** parameter types **are** enforced, because a method goes through overload selection [EXP-013]. Passing a `String` where `Integer` is declared is `No matching signature for function`.

```
interpreter  compiler/ObjClass.a24  FindOverload
conformance  TBD
```

[FUN-006] and [FUN-007] together mean the same annotation is decoration on a function and a contract on a method. See Annex D.

[FUN-008] A declared **return** type **is** enforced. `Exit` of a value that does not fit is `Type mismatch!`

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  TBD
```

11.3 Closures

[FUN-009] A nested subprogram captures the enclosing variables **by reference**, and the capture outlives the call that created it. A function returning a nested one hands back something that keeps reading and writing the same variable.

```
interpreter  compiler/ObjFunction.a24  ObjFunction
compiler     bootstrap/algol.c         alg_cell
conformance  TBD
```

[FUN-010] Each call to the enclosing subprogram creates a **fresh** set of captured variables. Two counters made the same way do not share a count.

```
interpreter  compiler/Environment.a24  Environment
compiler     bootstrap/algol.c         alg_closure
conformance  TBD
```

11.4 Subprograms as values

[FUN-011] A subprogram's name used without a call is a value. It may be assigned to a variable, passed as an argument, stored in a collection, and called from wherever it comes to rest.

```
interpreter  compiler/ObjFunction.a24  ObjFunction
compiler     bootstrap/algol.c         alg_call
unit         Interpret Local Function
conformance  TBD
```

11.5 Nesting

[FUN-012] Subprograms may be declared inside subprograms, to any depth.

 **compile-only divergence.** The C back end refuses more than one level with A function nested more than one level deep is not supported by the C back end yet. See Annex C.

```
interpreter  compiler/Parser.a24  ParseFunction
conformance  TBD
```

12. Classes and objects

12.1 Declaration

[CLS-001] A class is declared with an optional superclass, an optional header of sections, and a body.

```
ClassDecl = "class" identifier [ "(" identifier ")" ] ";"
           [ Sections ] "begin" { Member } "end" .
```

```
interpreter  compiler/Parser.a24  ClassDeclaration
unit         Parse Class Declaration
unit         Parse Class No Begin
conformance  TBD
```

[CLS-002] Fields are declared in var sections of the header [VAR-009]; methods in the body. Visibility markers apply to both [DCL-011].

```
interpreter  compiler/Parser.a24  ClassDeclaration
conformance  TBD
```

12.2 Construction

[CLS-003] An instance is made by **calling the class**: `Point(3, 4)` .

```
interpreter  compiler/Interpreter.a24  VisitCall
compiler     bootstrap/algol.c         alg_new
conformance  TBD
```

[CLS-004] A constructor is a member named `constructor Init` . Construction checks its arity, and a class with no constructor takes no arguments — `C(1, 2)` on such a class is Expected 0 arguments but got 2.

```
interpreter  compiler/ObjClass.a24  FindMethod
conformance  TBD
```

[CLS-005] ⚠ A field's initializer is evaluated **once per instance**, at construction. Two instances of a class whose field is `List := []` hold two different Lists.

```
interpreter  compiler/ObjClass.a24  SeedFields
compiler     bootstrap/algol.c      alg_class_field
conformance  TBD
```

[CLS-006] A field with no initializer begins as `nil` [VAR-002].

```
interpreter  compiler/ObjClass.a24  SeedFields
conformance  TBD
```

12.3 Members

[CLS-007] A field is read without parentheses and a method is called with them. There is no getter declaration — see [TYP-012].

```
interpreter  compiler/ObjInstance.a24  Get
unit         Call Setters And Getters
conformance  TBD
```

[CLS-008] Every instance answers `ClassName` , and does so **ahead of its fields**: the name belongs to the language, so a field of that name cannot take it.

```
interpreter  compiler/ObjInstance.a24  Get
compiler     bootstrap/algol.c         alg_property
conformance  TBD
```

[CLS-009] A class declaring `ToString()` decides how its instances render through `Str` and `print`.
With none, an instance renders as its class name followed by `instance` — `C instance`.

```
interpreter  compiler/Interpreter.a24  Stringify
conformance  TBD
```

[CLS-010] Reading or calling a member the class does not have is `Undefined` property `'X'`.

```
interpreter  compiler/ObjInstance.a24  Get
unit         Call Undefined Getter
conformance  TBD
```

12.4 Inheritance

[CLS-011] `class D (B)` makes `D` inherit `B`'s fields and methods. A method of the same name overrides the inherited one.

```
interpreter  compiler/ObjClass.a24  FindMethod
conformance  TBD
```

[CLS-012] `super.M()` calls the version above the class that declared the calling method, not above the runtime class.

```
interpreter  compiler/Interpreter.a24  VisitSuperExpr
compiler     bootstrap/algol.c         alg_invoke_from
conformance  TBD
```

[CLS-013] A class may not inherit from itself: A class can't inherit from itself.

```
interpreter  compiler/Resolver.a24  VisitClassStmt
unit         Inherit From Self
conformance  TBD
```

[CLS-014] A superclass must be a class. Naming a variable holding something else fails at construction with `Only instances have properties.`, and naming nothing at all with `Undefined variable 'X'`.

```
interpreter  compiler/Interpreter.a24  VisitClassStmt
unit         Inherit Not A Class
conformance  TBD
```

⚠ The first message describes neither the mistake nor the construct. See Annex D.

12.5 Objects

[CLS-015] object N; declares a class having exactly one instance, reached by the name itself. The instance is built on first use, so an object may refer to another declared later in the file.

```
interpreter  compiler/Parser.a24  ClassDeclaration
compiler    bootstrap/algol.c    alg_singleton
unit        An Object Takes Visibility Sections
conformance TBD
```

[CLS-016] An object is not callable. Config() is Can only call functions and classes.

```
interpreter  compiler/Interpreter.a24  VisitCall
conformance TBD
```

13. Enumerations

13.1 Declaration

[ENU-001] An enumeration is declared with type , naming its members in order.

```
EnumDecl = "type" identifier "=" "(" identifier { "," identifier } ")" ";" .
```

```
interpreter  compiler/Parser.a24  EnumDeclaration
conformance TBD
```

[ENU-002] Each member is bound as a **bare name** in the enclosing scope and is also reachable qualified as Type.Member . Both spellings denote the **same interned object**, so RED = Colour.RED is true.

```
interpreter  compiler/ObjEnum.a24  ObjEnumType
compiler    bootstrap/algol.c    alg_enum_member
unit        Both Spellings Of An Enum Member Type As The Enum
conformance TBD
```

[ENU-003] Because members bind as bare names, two enumerations may not share a member name: the second is 'A' is already defined.

```
interpreter  compiler/Interpreter.a24  VisitEnumStmt
conformance TBD
```

[ENU-004] Naming a member the type does not have is Undefined enum member 'X'.

```
interpreter  compiler/ObjEnum.a24  ObjEnumType
conformance  TBD
```

13.2 Values

[ENU-005] Members compare by **identity** [VAL-011]. A member of one enumeration is never equal to a member of another, whatever they are called.

```
interpreter  compiler/Interpreter.a24  IsEqual
unit        An Enum Member Does Not Satisfy Another Enum
conformance  TBD
```

[ENU-006] M is T is true for the member's own type and false for every other.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
unit        An Enum Type Name Types As Itself
conformance  TBD
```

[ENU-007] A member renders as its bare name: Str(RED) is RED .

```
interpreter  compiler/ObjEnum.a24  ToString
conformance  TBD
```

[ENU-008] Members are **not ordered**. RED < GREEN is Operands must be numbers.

```
interpreter  compiler/Interpreter.a24  VisitBinary
conformance  TBD
```

13.3 The ordinal

[ENU-009] ⚠ The **first member of every enumeration is falsey**, and every later member is truthy, because truthiness reads the member's position [VAL-008].

```
interpreter  compiler/Interpreter.a24  IsTruthy
compiler     bootstrap/algol.c         alg_truthy
conformance  TBD
```

[ENU-010] ⚠ A member is otherwise **opaque**. It has no reachable properties: RED.Ordinal is Only instances have properties. The position that decides [ENU-009] cannot be read by a program.

interpreter compiler/ObjEnum.a24 ObjEnum
conformance TBD

A program can discover whether a member is falsey only by testing it for truth. See Annex D.

14. Collections

14.1 Construction

[COL-001] A bracketed list of values is a `List`, and `[]` is an empty one. A bracketed list of `key : value` pairs is a `Map`, and `[:]` is an empty one.

interpreter compiler/Parser.a24 Primary
conformance TBD

[COL-002] `List()`, `Set()`, `Stack()` and `Map()` construct empty collections, `Set(L)` builds a `Set` from a collection, and `Array(N)` an `Array` of `N` elements.

interpreter compiler/Interpreter.a24 ArrayNative
compiler bootstrap/algol.c alg_array
conformance TBD

14.2 Members

[COL-003] Every collection answers `Length` and `IsEmpty` as properties, and `Contains` as a method. Everything else is per kind:

	List	Set	Stack	Array	Map
Length IsEmpty Contains	•	•	•	•	•
Get	•			•	•
Add	•	•			
Insert RemoveAt	•				
IndexOf Sort	•			•	
Clear	•	•	•		•
Remove		•			•

	List	Set	Stack	Array	Map
Set Fill				•	
Put Keys Values					•
ToList		•			
Push Pop Peek			•		

```

interpreter  compiler/ObjCollection.a24  Get
conformance  TBD

```

[COL-004] ⚠ A List has **no** Remove . Removing a value from a List means finding it with IndexOf and passing that to RemoveAt , while a Set and a Map remove by value and by key directly.

```

interpreter  compiler/ObjCollection.a24  Get
conformance  TBD

```

[COL-005] A member a kind does not have is Undefined property 'X' .

```

interpreter  compiler/ObjCollection.a24  Get
conformance  TBD

```

[COL-006] ⚠ A collection member's name is matched **exactly**. L.add(2) is Undefined property 'add' .

⚠ **compile-only divergence.** The C back end matches these names case-insensitively and accepts L.add(2) . See Annex C.

```

interpreter  compiler/ObjCollection.a24  Get
conformance  TBD

```

14.3 Order

[COL-007] Every collection iterates in **insertion order**, including Set and Map . This is specified rather than left to the representation, because both processors must produce the same output.

```

interpreter  compiler/ObjCollection.a24  Items
compiler    bootstrap/algol.c           alg_iterable
conformance  TBD

```


[COL-008] Re-assigning an existing Map key keeps the key's **original position**. Put on a key already present replaces the value and does not move it to the end.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler    bootstrap/algol.c           alg_put
conformance TBD
```

[COL-009] Keys() and Values() answer in that same order, so the two correspond element for element.

```
interpreter  compiler/ObjCollection.a24  Invoke
conformance TBD
```

14.4 Behaviour

[COL-010] A Set holds each value once. Adding a value it already has leaves its length unchanged.

```
interpreter  compiler/ObjCollection.a24  Invoke
conformance TBD
```

[COL-011] ⚠ Remove answers **different kinds of thing** by kind. A Map returns the value removed, and nil when the key was absent. A Set returns whether there was anything to remove.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler    bootstrap/algol.c           alg_remove
conformance TBD
```

[COL-012] Membership — Contains, in, and Map key lookup — compares strictly, without numeric promotion. See [VAL-013].

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler    bootstrap/algol.c           strict_equals
conformance TBD
```

[COL-013] Sort orders in place and is **stable**. It orders numbers against numbers and text against text; mixing them is Can only sort numbers against numbers, or text against text.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler    bootstrap/algol.c           alg_sort
conformance TBD
```

[COL-014] Subscripting reads by position for a List and an Array and by key for a Map. A Set has no positions and is refused with Subscript target should be an ordinal.

```
interpreter  compiler/ObjCollection.a24  At
compiler    bootstrap/algol.c           alg_subscript_get
conformance  TBD
```

[COL-015] An `Array` is fixed in size and does not grow on assignment — see [TYP-008].

```
interpreter  compiler/ObjCollection.a24  At
conformance  TBD
```

15. Modules

15.1 Importing

[MOD-001] `uses` imports another file. A bare identifier names the file of that name with `.a24` appended; a quoted string is a path.

```
UsesStmt = "uses" ( identifier | string_lit ) ";" .
```

```
interpreter  compiler/Parser.a24  UsesStatement
conformance  TBD
```

[MOD-002] A module is looked for **beside the importing file first**, then in the working directory. Two directories may therefore hold files of one name without either reaching the other's. Failure is `Could not find module 'X': no X.a24 in ...`

```
interpreter  compiler/Parser.a24  ResolveModule
conformance  TBD
```

[MOD-003] A module is loaded and executed **once**, keyed by its resolved path, however many files import it. A second import of the same file sees the names without re-running the body.

```
interpreter  compiler/Parser.a24  UsesStatement
conformance  TBD
```

[MOD-004] A file may open with `unit N; .` If present, `N` must match the file's own name: `Unit 'Wrong'` must match its file name `'Mismatch'`.

```
interpreter  compiler/Parser.a24  UnitHeader
conformance  TBD
```

15.2 Exports

[MOD-005] A module exports its top-level declarations, except those marked `private` .

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance  TBD
```

[MOD-006] At the top level of a module, `private` precedes a **single declaration** and hides it. It is not a section marker there, unlike inside a class [DCL-011].

```
interpreter  compiler/Parser.a24  RecordPrivate
unit          Module Private Is Not A Section Marker
conformance  TBD
```

[MOD-007] A private name is invisible to an importer both bare and qualified. Qualified, it is `Undefined` name `'Hidden'` in unit `'Mid'`.

```
interpreter  compiler/Interpreter.a24  Qualified
conformance  TBD
```

[MOD-008] Two imported modules exporting one name is refused: `'Clash'` is already defined; mark it `private` in one of the modules.

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance  TBD
```

15.3 Visibility

[MOD-009] ⚠ `uses` is **not transitive**. If `A` imports `B` and `B` imports `C` , then `A` does not see `C` 's names — `B` may use them, and `A` may not. The diagnostic names the unit that would export it:

```
Undefined variable 'DeepName'. Unit 'Deep' exports it; this file has no 'uses' for it.
```

```
interpreter  compiler/Interpreter.a24  LookupVariable
conformance  TBD
```

[MOD-010] An exported name may be qualified by its unit — `Mid.MidName()` — and the qualifier is resolved statically as a unit rather than evaluated as a value.

```
interpreter  compiler/Interpreter.a24  Qualified
conformance  TBD
```

[MOD-011] `System` is the unit of the built-in functions. No file imports it and every file may qualify against it: `System.Copy('abcdef', 0, 3)` .

```
interpreter  compiler/Resolver.a24  Units
conformance  TBD
```

15.4 Cycles

[MOD-012] ⚠ **Circular imports do not work**, and fail differently depending on the shape:

- Between modules — `A` uses `B` and `B` uses `A` — the program fails with `Type mismatch!` , a diagnostic having nothing to do with types.
- Through the **root** file — a module importing the file that is being run — the root's own imports fail with `Undefined variable 'X'` . even though the root's body has already run.

⚠ **compile-only divergence.** A cycle through the root also refuses to compile, with `Two modules named 'X' is not supported by the C back end yet.` See Annex C, C-1.

```
interpreter  compiler/Parser.a24  UsesStatement
conformance  TBD
```

See Annex D.

16. Built-in functions

16.1 The set

[RT-001] Twenty-five names are built in. Twenty-two are always available:

Length	Copy	Pos	Str	Ord	Char	Val
Max	Mod	clock				
List	Set	Stack	Array	Map	Buffer	
TextFile		FileExists				
ParamCount		ParamStr				
Write	WriteLn					

```
interpreter  compiler/Interpreter.a24  Bultins
conformance  TBD
```

[RT-002] The remaining three — `AssertTrue` , `AssertEqual` and `Fail` — are registered **only while** `--test` **is running** [see 19]. Calling one outside a test run is `Undefined variable 'AssertTrue'`.

```
interpreter  compiler/Interpreter.a24  RunTests
conformance  TBD
```

16.2 Text

[RT-003] ⚠ `Length(V)` **stringifies its argument and measures the text**, in bytes. It is not a collection's count: for a List of three elements `Length(L)` is the length of `[10, 20, 30]` — twelve — while `L.Length` is three.

```
interpreter  compiler/Interpreter.a24  LengthNative
compiler     bootstrap/algol.c         alg_length
conformance  TBD
```

Two different things are spelled `Length` , and the wrong one returns a plausible number rather than an error. See Annex D.

[RT-004] `Copy(Text, Begin, Length)` takes a substring, counting from zero. The length is clamped to what remains, so `Copy('abcdef', 3, 99)` is `def` . A start outside the text is `Copy` failed: `Start -2` out of range `0..6`.

```
interpreter  compiler/Interpreter.a24  CopyNative
compiler     bootstrap/algol.c         alg_copy
conformance  TBD
```

[RT-005] `Pos(Text, Part)` answers the zero-based index of `Part` within `Text` , or `-1` when it is absent.

```
interpreter  compiler/Interpreter.a24  PosNative
compiler     bootstrap/algol.c         alg_pos
conformance  TBD
```

[RT-006] `Str(V)` renders any value: an Integer bare, a Double always with a point (`1.0`), a Boolean lowercase (`true`), `nil` as `nil` , a List as `[10, 20, 30]` , a Map as `[1:2]` , and an instance by its `ToString` [CLS-009].

interpreter	compiler/Interpreter.a24	Stringify
compiler	bootstrap/algol.c	alg_str
conformance	TBD	

[RT-007] `Ord(C)` answers the code point of a single character, as an **Integer**. Anything longer is `Ord` failed: `'ab'` has no ordinal.

interpreter	compiler/Interpreter.a24	OrdNative
compiler	bootstrap/algol.c	alg_ord
conformance	TBD	

[RT-008] `Char(N)` answers the character with code point `N`, limited to 0 ... 127 [LEX-025]. `Ord` and `Char` are inverse within that range.

interpreter	compiler/Interpreter.a24	CharNative
compiler	bootstrap/algol.c	alg_char
conformance	TBD	

16.3 Numeric

[RT-009] ⚠ `Val(S)` parses a number from text and **always yields a Double**, even where the text is integral: `Val('42')` is `42.0` and `Val('42') is Integer` is false. Failure is `Val` failed: `'abc'` is not a number.

interpreter	compiler/Interpreter.a24	ValNative
compiler	bootstrap/algol.c	alg_val
conformance	TBD	

[RT-010] ⚠ `Max(A, B)` takes **Integers only**. `Max(3.5, 2)` is `Max` expects Integers.

interpreter	compiler/Interpreter.a24	MaxNative
compiler	bootstrap/algol.c	alg_max
conformance	TBD	

[RT-011] `Mod(A, B)` answers the remainder, whose sign follows the dividend: `Mod(-7, 3)` is `-1`. A zero divisor is `Mod` failed: Division by zero.

interpreter	compiler/Interpreter.a24	ModNative
compiler	bootstrap/algol.c	alg_mod
conformance	TBD	

[RT-012] `clock()` answers the seconds since the epoch as a **Double**, at millisecond resolution.

interpreter	compiler/Interpreter.a24	ClockNative
compiler	bootstrap/algol.c	alg_clock
conformance	TBD	

16.4 Environment

[RT-013] `ParamStr(0)` is the program's own name and `ParamCount()` does not count it, so a program run with no arguments reports zero.

interpreter	compiler/Interpreter.a24	ParamCountNative
compiler	bootstrap/algol.c	alg_param_count
conformance	TBD	

[RT-014] `FileExists(Name)` answers whether the named file exists.

interpreter	compiler/Interpreter.a24	FileExistsNative
compiler	bootstrap/algol.c	alg_file_exists
conformance	TBD	

[RT-015] `Write(V)` and `WriteLn(V)` write the stringified value to standard output, `WriteLn` following it with `#10` — always that byte, never the host's line separator, so one program writes the same bytes everywhere.

interpreter	compiler/Interpreter.a24	WriteLnNative
compiler	bootstrap/algol.c	alg_writeln
conformance	TBD	

17. Program initialization and execution

17.1 A program

[INI-001] A program is one file. Its top-level statements are executed in the order they are written, and there is no distinguished entry point — no `main`, and no statement that begins execution.

interpreter	compiler/Main.a24	Run
conformance	TBD	

[INI-002] A declaration takes effect when its statement is reached, so a name is undefined above its declaration — see [DCL-006].

```
interpreter  compiler/Interpreter.a24  Interpret
conformance  TBD
```

17.2 Module initialization

[INI-003] A `uses` loads and runs its module **at the point it appears**. Root statements written between two `uses` clauses therefore run between the two module bodies:

<code>WriteLn ('1 root');</code>	<code>→ 1 root</code>
<code>uses Alpha;</code>	<code>→ Alpha body</code>
<code>WriteLn ('2 root');</code>	<code>→ 2 root</code>
<code>uses Gamma;</code>	<code>→ Gamma body</code>
<code>WriteLn ('3 root');</code>	<code>→ 3 root</code>

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance  TBD
```

[INI-004] A module is initialized once [MOD-003], and its imports are initialized before it, so a module's own body may use anything it imported.

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance  TBD
```

 **compile-only divergence, and a silent one.** The compiled program runs **every** module initializer before **any** root statement, so the example above prints both module bodies first and then all three root lines. The same program produces two different orders. See Annex C, C-5.

17.3 Termination

[INI-005] A program that reaches the end of its statements exits with status **0**.

```
interpreter  compiler/Main.a24  Main
conformance  TBD
```

[INI-006] Every failure the language reports exits with status **70** — an uncaught `raise` [STM-021], and equally a scan, parse, resolution or type error, which are reported before any statement runs.

```
interpreter  compiler/Main.a24  CheckScanned
compiler    bootstrap/algol.c  alg_error
conformance  TBD
```


⚠ 70 does not distinguish *a program that failed* from *a program that was never run*. A caller wanting to tell a compile error from a runtime one must read the diagnostic.

17.4 Arguments

[INI-007] A program reads its command line through `ParamCount` and `ParamStr` [RT-013].

`ParamStr(0)` is the program's own name, and arguments follow from index 1.

interpreter	compiler/Main.a24	ArgumentsFrom
compiler	bootstrap/algol.c	alg_set_arguments
conformance	TBD	

18. Errors

18.1 Phases

[ERR-001] A program passes through five phases: **scan**, **parse**, **resolve**, **check**, **run**. The first four complete over the whole program — its imports included — before any statement is executed.

interpreter	compiler/Main.a24	Run
conformance	TBD	

[ERR-002] An error in any of the first four phases means **no statement runs at all**. A program cannot produce output and then fail to compile.

interpreter	compiler/Main.a24	Run
conformance	TBD	

[ERR-003] A runtime error occurs during execution. Statements before it have run and their output stands.

interpreter	compiler/Interpreter.a24	Interpret
conformance	TBD	

18.2 Diagnostics

Diagnostics are part of the observable surface [1.2]; their wording and shape are specified.

[ERR-004] A **scan** error reads `[line N] Error: <message>` and carries no source excerpt.

⚠ It is **recorded rather than raised**: the scanner sets a flag and keeps the message, and a driver must ask. A driver that does not ask will scan a damaged token stream and carry on.

```
interpreter  compiler/Scanner.a24  HadError
unit         Scan Unrecognized Character Is Recorded
conformance  TBD
```

[ERR-005] A **parse** or **resolution** error prints the message and a three-line excerpt naming the file, the line, and the offending token:

```
Uncaught: Expect variable name.
[ERROR] e.a24: Expect variable name.
[ERROR] 2 | var := 1;
[ERROR]   | ^^
```

```
interpreter  compiler/Console.a24  Error
conformance  TBD
```

[ERR-006] ⚠ A **type** error prints only:

```
Uncaught: Type mismatch!
```

It names no file, no line, no token and neither of the types involved, and the same five words are used for every mismatch the checker finds.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  TBD
```

See Annex D.

18.3 Catching

[ERR-007] Only **runtime** errors are catchable, and they are caught as a String carrying the diagnostic [STM-020].

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
conformance  TBD
```

[ERR-008] A `try` around a scan, parse, resolution or type error catches nothing, because those phases complete before the `try` is reached. Wrapping a mistyped declaration in a handler does not suppress it.

```
interpreter  compiler/Main.a24  Run
conformance  TBD
```

18.4 Status

[ERR-009] Every failure exits with status **70**, whichever phase reported it [INI-006].

```
interpreter  compiler/Main.a24  Main
conformance  TBD
```

19. Test blocks

A test block is a declaration that a test run executes and an ordinary run ignores. The report it produces is specified line for line, because it is the surface on which two implementations are compared.

19.1 Declaration

[TST-001] A test is written `test` followed by a string literal naming it, then a block.

```
TestDecl = "test" string_lit ";" Block .
```

`test` is not a keyword [LEX-011]; it is recognised here by the quoted name that follows it, so a variable may still be called `test` .

```
interpreter  compiler/Parser.a24  Declaration
conformance  TBD
```

[TST-002] A test block is a declaration and does not run when the program runs.

```
interpreter  compiler/Interpreter.a24  HoistTests
conformance  TBD
```

19.2 Running

[TST-003] A test run executes the test blocks **instead of** the program. The top-level statements do not run — only the declarations they would have created.

```
interpreter  compiler/Interpreter.a24  RunTests
conformance  TBD
```

[TST-004] Tests are collected from the root file and from every module it reaches, each file contributing once however many ways it is reached.

```
interpreter  compiler/Interpreter.a24  HoistTests
conformance  TBD
```

[TST-005] Tests are reported **sorted by name within a file**, and files in the order their first test was met — which for `uses` is load order. Source order within a file is not preserved.

```
interpreter  compiler/Interpreter.a24  RunTests
conformance  TBD
```

[TST-006] A program's own `Write` and `WriteLn` output is **swallowed** during a test run, so it cannot interleave with the report.

```
interpreter  compiler/Interpreter.a24  SuppressOutput
compiler     bootstrap/algol.c         alg_test_begin
conformance  TBD
```

[TST-007] A value raised inside a test body and not caught makes that test **fail**; it does not end the run, and later tests still execute.

```
interpreter  compiler/Interpreter.a24  RunTests
compiler     bootstrap/algol.c         alg_test_run
conformance  TBD
```

19.3 The report

[TST-008] The report consists of these lines, in this order:

```
[INFO] Running N tests...
[INFO] < file >
[INFO] Test: <name> <leader> [ PASS ]
[ERROR] <file>: <message>          only after a FAIL, and only interpreted
[INFO]
[INFO] All N tests passed.
```

A file's block is opened by its `< file >` line, and a blank `[INFO]` line separates files and precedes the summary.

```

interpreter  compiler/Interpreter.a24  Report
compiler     bootstrap/algol.c        alg_test_run
conformance  TBD

```

[TST-009] The dot leader is 55 – Length(name) dots, clamped to a minimum of one, so a name longer than the banner still produces a well-formed line.

```

interpreter  compiler/Interpreter.a24  Report
compiler     bootstrap/algol.c        alg_test_run
conformance  TBD

```

[TST-010] The report is coloured, and the colours are part of it: the [INFO] tag white and blue, [ERROR] white and red, the file name cyan, PASS green, FAIL red, and the summary green when all passed and red otherwise.

⚠ The escapes are emitted **unconditionally**, whether or not the output is a terminal — the language has no way to ask — so anything reading a report strips or transliterates them.

```

interpreter  compiler/Console.a24  INFO
compiler     bootstrap/algol.c     INFO_TAG
conformance  TBD

```

[TST-011] The summary is All N tests passed. or N of M tests failed. , and the run exits **0** when every test passed and **70** when any failed.

```

interpreter  compiler/Interpreter.a24  RunTests
compiler     bootstrap/algol.c        alg_test_summary
conformance  TBD

```

19.4 Assertions

[TST-012] Three assertions exist, and only during a test run [RT-002]:

Call	Message on failure
AssertTrue(V)	Assertion 'left = right' failed.
AssertEqual(E, A)	Assertion 'left = right' failed. Expected 'E' but got 'A'.
Fail(M)	Failed. M

⚠ Two spaces follow the full stop in the second and third, and AssertTrue reports a comparison it did not make.

interpreter	compiler/Interpreter.a24	AssertTrueNative
compiler	bootstrap/algol.c	alg_assert_equal
conformance	TBD	

[TST-013] `AssertEqual` compares with `=` [VAL-009], so it promotes numerically and holds a `Char` unequal to a `String` [LEX-026].

interpreter	compiler/Interpreter.a24	AssertEqualNative
conformance	TBD	

19.5 Compiled runs

[TST-014] ⚠ compile-only divergence. A compiled report omits the [ERROR] line after a failure, because compiled code has no line information to build one from. Every other line is byte-identical, colour included. See Annex C, C-3.

compiler	bootstrap/algol.c	alg_test_run
conformance	TBD	

Annex A — grammar summary (*non-normative*)

Every production stated in the chapters, collected. This annex adds nothing: each line appears in the chapter that specifies it, and `spec/spec.sh` checks that none has been added here or lost from here.

⚠ The grammar is partial, deliberately. Where a construct's shape was verified by running it rather than by writing a production, the chapter states it in prose and no production appears below. The gaps are named at the end of this annex rather than filled with plausible-looking rules, because a production nobody checked is exactly the kind of claim this specification exists to avoid.

Lexical

```

letter      = "a" ... "z" | "A" ... "Z" | "_" | "?" .
decimal_digit = "0" ... "9" .

identifier   = letter { letter | decimal_digit } .

integer_lit  = decimal_digit { decimal_digit } .
double_lit   = decimal_digit { decimal_digit } "." decimal_digit { decimal_digit } .
char_lit     = "'" source_byte "'" | "#" decimal_digit { decimal_digit } .
string_lit   = '"' { source_byte_other_than_quote | "'" } '"' .

```

Declarations

```
VarDecl    = "var" identifier [ ":" Type ] [ "!=" Expression ] ";" .
ConstDecl  = "const" identifier [ ":" Type ] "!=" Expression ";" .
Type       = identifier [ "of" identifier ] .

VarSection = "var" { identifier { "," identifier } [ ":" Type ]
               [ "!=" Expression ] ";" } .

FunDecl    = ( "function" | "procedure" ) identifier "(" [ Params ] ")"
               [ ":" Type ] ";" [ Sections ] Block .
Params     = identifier [ ":" Type ] { "," identifier [ ":" Type ] } .

ClassDecl  = "class" identifier [ "(" identifier ")" ] ";"
               [ Sections ] "begin" { Member } "end" .
EnumDecl   = "type" identifier "=" "(" identifier { "," identifier } ")" ";" .

UsesStmt   = "uses" ( identifier | string_lit ) ";" .
TestDecl   = "test" string_lit ";" Block .
```

Statements

```
CaseStmt = "case" Expression "of" { Arm } [ "else" Statement ] "end" .
Arm      = Expression { "," Expression } ":" Statement .
```

What is specified in prose instead

These constructs are specified by rule and verified by probe, without a production:

Construct	Specified by
Operator precedence and associativity	[EXP-001], [EXP-002] — a table, derived by running distinguishing cases
if , while , counted for , for ... in	[STM-003] ... [STM-007]
break , Exit , raise , print	[STM-010], [STM-014], [STM-016], [STM-022]
try / except / on handlers	[STM-017] ... [STM-019]
Blocks and expression statements	[STM-001], [STM-002]
Visibility sections	[DCL-011], [DCL-012]

⚠ Sections , Block , Member and Statement are referenced above and not themselves defined. Completing the grammar is worth doing; inventing those four productions from memory is not.

Annex B — index of built-in functions (*non-normative*)

The twenty-five built-in names, with the rule specifying each. `spec/spec.sh` checks this list against the names the interpreter actually registers.

Name	Rule	Summary
AssertEqual	[TST-012]	Fails unless two values are equal; test runs only
AssertTrue	[TST-012]	Fails unless a value is truthy; test runs only
Fail	[TST-012]	Fails outright with a message; test runs only
Array	[COL-002]	An Array of N elements, each <code>nil</code>
Buffer	[E.2]	Growable bytes with an explicit lifetime
List	[COL-002]	An empty List
Map	[COL-002]	An empty Map
Set	[COL-002]	An empty Set, or a Set of a collection's values
Stack	[COL-002]	An empty Stack
Char	[RT-008]	The character with a code point, 0 ... 127
Copy	[RT-004]	A substring, from a zero-based start, length clamped
Length	[RT-003]	⚠ The length of the argument's text , not a count
Ord	[RT-007]	The code point of one character, as an Integer
Pos	[RT-005]	A zero-based index, or -1 when absent
Str	[RT-006]	Any value rendered as text
Val	[RT-009]	⚠ A number parsed from text, always a Double
Max	[RT-010]	⚠ The greater of two Integers only
Mod	[RT-011]	The remainder, its sign following the dividend
clock	[RT-012]	Seconds since the epoch, as a Double
FileExists	[RT-014]	Whether a named file exists
TextFile	[E.2]	A text file handle
ParamCount	[RT-013]	The argument count, not counting the program
ParamStr	[RT-013]	An argument by index; 0 is the program

Name	Rule	Summary
Write	[RT-015]	Writes a value
WriteLn	[RT-015]	Writes a value and #10

⚠ `clock` is the only built-in spelled in lower case, and the only one whose name is not a noun or an imperative. Nothing depends on this; it is noted because a reader will wonder.

Annex C — compiler divergences (*non-normative*)

Where the C back end does not do what the interpreter does. The interpreter is the authority [1.1], so every entry here is a defect in the compiler rather than a choice the language has made.

⚠ A **loud** divergence refuses to compile and says why. A **silent** one produces a program that runs and behaves differently. The second kind is far worse, and the column says which each is.

C-1 — A file in an import cycle with the root will not compile. (*loud*)

The root file is never entered in the parser's `Loaded` map, so a module importing the root back parses it a second time, and the emitter then sees two units of one name:

```
Two modules named 'Parser' is not supported by the C back end yet.
```

`compiler/Parser.a24` uses `Interpreter`, which uses `Parser`, so the compiler's own source is an instance. The same file runs correctly interpreted. This is currently the only known case of a valid program having no compiled form.

C-2 — Functions may not nest more than one level deep. (*loud*) (*refers to [FUN-012]*)

```
A function nested more than one level deep is not supported by the C back end yet.
```

Three levels of nesting run correctly interpreted and refuse to compile.

C-3 — A compiled assertion failure carries no message. (*loud enough*)

`alg_test_run` prints no `[ERROR]` line, because compiled code has no line information to put in one. The `FAIL` stands alone where the interpreter also prints the assertion and a caret. A report comparison drops those lines for that reason.

C-4 — Collection member names are matched case-insensitively. (*silent*) (*refers to [COL-006]*)

```
var L := [1];
L.add (2);
```

Interpreted this is `Undefined property 'add'`. Compiled it works, and the program prints a length of 2.

⚠ This is the first **silent** divergence recorded, and it runs in the more dangerous direction: the compiler **accepts a program the language refuses**. A program developed against the compiler can use lowercase member names throughout and fail everywhere the moment it is run interpreted, with no warning from either processor that the two disagree.

The C runtime compares collection members with `alg_stricmp`, which is deliberate — but `bootstrap/algol.h` asserted alongside it that this was "what the interpreter does", and it is not. That comment has been corrected.

The interpreter is the authority [1.1], so the fault is the compiler's.

Fix: compare exactly in `alg_property` and `alg_invoke` for collection receivers. Note that `Length` and `IsEmpty` are matched case-insensitively by the same code, so the change must cover those too, and any program relying on the looser spelling will stop compiling — which is the point.

C-5 — Module bodies run at a different time. (*silent*) (refers to [INI-003], [INI-004])

Interpreted, a `uses` runs its module where it appears, so root statements interleave with module bodies in source order. Compiled, every module initializer runs before any root statement.

<code>WriteLn ('1 root');</code>	interpreted	compiled
<code>uses Alpha;</code>	1 root	Alpha body
<code>WriteLn ('2 root');</code>	Alpha body	Gamma body
<code>uses Gamma;</code>	2 root	1 root
<code>WriteLn ('3 root');</code>	Gamma body	2 root
	3 root	3 root

⚠ Silent, and unlike C-4 it needs no unusual spelling to provoke: any program whose modules print, open a file, or set a variable the root then reads will behave differently under the two processors, and nothing warns.

The compiled shape follows from how the C is emitted — `main` calls each `init_<Unit>()` and then `init_Main()` — and it is the easier order to produce, since a module's initializer is a function and the root's body is not special.

The interpreter is the authority [1.1], so the compiler is wrong.

Fix: emit the root's top-level statements in place, interleaved with the `init_<Unit>()` calls that correspond to its `uses`, rather than collecting them all into `init_Main()` after the fact. That is a change to how the root unit is emitted and not to the runtime.

Not a divergence, and worth stating as a requirement: interpreted and compiled `--test` reports are byte-identical, colour included — 239 lines and 1,416 escape sequences for the full suite. Any difference between them is a defect in one or the other.

Annex D — advisory notes (*non-normative*)

Where the specified behaviour looks like a mistake. Nothing here weakens the rule it refers to: the body states what the language does, and this annex argues about it. Entries are added as the chapters that expose them are written.

D-1 — Integer overflow is silent. (*refers to [LEX-018]*)

A literal or an arithmetic result outside the signed 32-bit range wraps with no diagnostic, so `2147483648` is `-2147483648` and `9999999999999999` is `276447231`. A program can compute a wrong answer and give no sign of it.

Both processors wrap identically, so this is at least consistent, and the wrapping is what C does natively — making it free in the compiler and deliberate work to detect. Changing it would mean either a check on every arithmetic operation or a widened Integer, and it would break any program relying on the wrap.

Recommended: raise on overflow. A language whose compiler is written in itself cannot afford a silent wrong answer in its own arithmetic, and the cost is a branch on operations that are not this language's bottleneck.

D-2 — `?` alone is a valid identifier. (*refers to [LEX-008]*)

Because `?` is classed as a letter so that `Gate?` scans as one word, it may also *begin* an identifier, and `var ? := 7;` declares a variable named `?`. That is almost certainly not intended; it falls out of the letter class rather than from any decision.

Recommended: keep `?` as a trailing character only — a letter for the purpose of continuing an identifier, not of starting one. No plausible program relies on the current rule, and the change is confined to `IsAlpha` and its caller.

D-3 — `#0` is constructible but unstorable. (*refers to [LEX-032]*)

`#0` yields a Char, and a Char is a value like any other, but putting one into a String silently truncates it. The String type is C's NUL-terminated representation showing through, and the language does not say so anywhere.

Recommended: either refuse `#0` at construction, as [LEX-025] already refuses 128 and above, or give String an explicit length so it can hold a zero byte. Refusing is much the smaller change and matches the existing range check; storing it honestly is the better language.

D-4 — Widening is refused as firmly as narrowing. (*refers to [VAR-004]*)

`var X : Integer := 1.5;` should certainly be refused: the value does not fit. But `var X : Double := 1;` is refused on the same terms, and there the value fits exactly and every arithmetic operation in the language already promotes an Integer to a Double when the two meet. A programmer who writes `: Double` and initializes with `0` is told the types do not match.

The rule is at least symmetric and easy to state, and any relaxation has to answer what `var X : Double := 1;` then makes `X` — a Double holding 1.0, or an Integer that a declaration lied about.

Recommended: permit Integer where Double is declared, converting at the point of assignment, and leave narrowing refused. It matches what the arithmetic already does, and the alternative teaches that a declared type means something narrower than the operators do.

D-5 — `const` promises less than it appears to. (refers to [VAR-014])

`const C := V;` is legal where `V` is a variable, so a constant's value need not be known before the program runs. The word means only that the binding cannot be reassigned. A reader who takes `const` to mean a compile-time constant — as Pascal's does, and as most languages' do — will be wrong about when the initializer runs and about what the compiler can assume.

Nothing here is broken, and the run-time form is genuinely useful for a value computed once at startup.

Recommended: keep the behaviour and say so plainly in the language's own documentation, which currently does not. If a true compile-time constant is wanted later it needs a different word, not a narrowing of this one.

D-6 — Equality and membership disagree. (refers to [VAL-009], [VAL-013])

`1 = 1.0` is true, and `1 in [1.0]` is false. A Map holding the key `1` does not contain `1.0`. Each rule is defensible alone — `=` promotes because arithmetic does, and membership is strict because a hash table cannot be built over a relation that promotes — but together they mean a program can hold two values it calls equal and find only one of them in a collection.

`bootstrap/algol.c` already names this a rough edge in its own comments, which is a fair sign it was noticed and not resolved.

Recommended: make `in` and `Contains` follow `=`, and pay for it by hashing an Integer and a Double with the same numeric value to the same bucket. The alternative — making `=` strict — is a far larger change and would surprise every program doing ordinary arithmetic.

D-7 — `as` is an unchecked assertion. (refers to [VAL-007])

`X as Integer` where `X` holds `'text'` yields `'text'` and raises nothing. The cast silences the checker and is never verified, so the one construct a programmer reaches for when they know more than the checker does is also the one that cannot tell them when they are wrong.

The gradual type system needs an escape hatch, and an unchecked one is free where a checked one costs a test at every cast.

Recommended: check it at run time and raise on failure. The cost falls only on programs that use `as`, which are the programs that asked for the assurance; and a cast that cannot fail is not an assurance at all.

D-8 — Empty is truthy. (*refers to [VAL-008]*)

`0` is falsey, but `0.0`, `''`, `[]` and `[]:` are all truthy. So `if not S then` does not mean what a reader coming from most languages will expect it to mean, and `if S.Length = 0 then` is the only reliable spelling.

The rule is at least short to state, and it makes truthiness independent of a value's contents — a collection is a thing, and a thing is there.

Recommended: keep it, and say so loudly wherever the language is taught. The inconsistency worth fixing is Integer `0`, which is the odd one out: were `0` truthy too, the rule would be the genuinely simple "only `nil` and `False` are false". That change would break existing programs, so it is a language-version question rather than a defect to be quietly repaired.

D-9 — Visibility is advisory. (*refers to [DCL-015]*)

`private:` is checked by the type checker, and only where the receiver's type is known. `var C : Any := Counter(); C.Count` yields the private field and raises nothing. A subclass reading its parent's private member by bare name gets it, because a bare name inside a method resolves through `this`, and `this` reduces to no type at all — while the same member through a receiver declared as the parent is correctly refused.

So the guarantee is real for code that annotates its types and absent for code that does not, which is the opposite of where a guarantee is most wanted. It is also not a security property and was never meant to be: the checker is advisory by design in a gradually typed language.

⚠ Any fix collides with a deliberate decision recorded in the checker: `this` reduces to nothing precisely so that a class's own code is free of the visibility rule. Typing `this` as its class would make the bare-name case work and would also make every `this.Private` inside a subclass an error the language currently allows.

Recommended: enforce it at run time on the instance, where the class is always known, and leave the static check as the early warning it already is. Failing that, say plainly in the language's documentation that `private:` is a convention the checker helps with rather than a boundary.

D-10 — Integer division by zero raises; Double division by zero does not. (*refers to [EXP-006]*)

`1 / 0` is the runtime error `Division by zero`. `1.0 / 0` is `Infinity`, and `0.0 / 0` is `NaN`, and neither stops the program. So whether dividing by zero is a bug or a value depends on which numeric type reached the operator — and [EXP-005] means an Integer becomes a Double whenever it meets one, so the same expression can change category with an edit far from it.

The Double behaviour is IEEE 754 and is what C does for free; the Integer behaviour has no such answer available, since there is no integer infinity to produce.

Recommended: keep both. They are each correct for their type, and the alternatives are worse — raising on Double division would depart from IEEE for no gain, and returning a value for Integer division would have to invent one. This belongs in the language's documentation rather than in its defect list, and is recorded here only because the asymmetry is genuinely surprising.

D-11 — A parameter's declared type is decoration on a function and a contract on a method. (*refers to [FUN-006], [FUN-007]*)

`function F(N : Integer)` accepts a String, a Double or a Boolean without complaint. The same signature as a method refuses all three with `No matching signature for function.` Nothing in the language says the two differ, and the annotation looks identical in both places.

The cause is structural rather than deliberate: a method goes through overload selection, which compares whole signatures, and a top-level function does not overload [see 8.3] so nothing ever compares its parameters. Only the arity is checked.

⚠ Note the direction of the surprise. A declared return type IS enforced [FUN-008], so within one declaration the result is checked and the arguments are not — which is the reverse of what a reader would guess, since arguments come from outside and are the less trustworthy of the two.

Recommended: check parameter types on every call, not only where an overload has to be chosen. The check already exists in `Fits`; what is missing is calling it when there is nothing to select between. Until then, a type annotation on a top-level parameter should be read as a comment.

D-12 — Inheriting from a non-class reports the wrong thing. (*refers to [CLS-014]*)

```
var X := 1;
class C (X);
begin
end
```

fails with `Only instances have properties.` — a sentence naming neither the class, nor the superclass, nor inheritance, and describing a property access the program never wrote. The check happens where the superclass is used rather than where it is declared, so the message belongs to the machinery rather than to the mistake.

Diagnostics are part of the observable surface [1.2], so this is a specified behaviour and not merely a rough edge — a conforming implementation must reproduce the misleading sentence exactly.

Recommended: refuse it at the declaration with the shape the other inheritance errors already use — `'X' is not a class.` — which sits beside `A class can't inherit from itself.` and needs no new machinery. This is the cheapest entry in this annex to act on and the one a newcomer is likeliest to hit.

D-13 — Truthiness reads a value a program cannot. (*refers to [ENU-009], [ENU-010]*)

The first member of every enumeration is falsey, because truthiness reads the member's ordinal — and the ordinal is not reachable from the language. `RED` is false and `GREEN` is true, and a program can discover which is which only by testing each for truth.

So if `Colour` then depends on where a member sits in a list that the program cannot inspect, and reordering an enumeration's members silently changes the truth of every conditional written over it. Nothing warns, because nothing about the declaration looks conditional.

The rule exists so that enumerations behave like the small integers they are represented by, which is a real convention and not an accident.

Recommended: two things, and the first is worth doing whichever way the second goes. **Expose the ordinal**, so the value that governs the behaviour can at least be read and compared; `Ordinal` already exists on the implementation's own class and simply is not published. **Then reconsider [ENU-009] itself** — an enumeration member is not a number, nothing else in the language makes a declared name falsey by position, and a first member that is false is a trap laid for whoever adds a member at the front.

D-14 — Circular imports fail, and say something else. (*refers to [MOD-012]*)

A cycle between two modules fails with `Type mismatch!` — a sentence about types for a program whose types are fine. A cycle through the root file fails with `Undefined variable 'B'`. for a name the root plainly imports, and only after the root's own body has already run. Compiled, the same cycle refuses with `Two modules named 'X'`, which at least names the right subject.

Three shapes of one problem, three diagnostics, none of which says "these modules import each other".

Cycles are not obviously *wrong* to reject: a language that loads and executes each module once, in order, has a genuine question about what a half-initialized module should expose, and refusing is a defensible answer. But refusing is not what happens — the failures come from machinery downstream noticing that something is missing.

Recommended: detect the cycle where it happens, in the parser's `uses` handling, and refuse it by name — `Circular import: 'A' uses 'B' uses 'A'`. The loader already keeps the map that would make this a few lines. Whether cycles should later be *supported* is a separate and much larger question; being told about them plainly is worth having either way.

⚠ This is not hypothetical for this repository. `compiler/Parser.a24` uses `Interpreter`, which uses `Parser`, so the compiler's own source contains a cycle — which is why it cannot be compiled by itself [Annex C, C-1].

D-15 — Two different things are spelled `Length`. (*refers to [RT-003]*)

```
var L := [10, 20, 30];
L.Length      → 3
Length(L)     → 12
```

The property answers the collection's count. The function stringifies its argument and measures the text, so `Length(L)` is the length of `[10, 20, 30]`. `LengthNative` is literally `Exit Length(Str(Arguments[0]))`.

⚠ The failure mode is the bad one: the wrong call returns a **plausible number** rather than an error. A program asking `Length(L)` of a collection gets an answer, uses it, and is wrong — and for a List of one-digit numbers the two even coincide at small sizes before diverging.

The function is right for its intended argument. `Length('abc')` is 3, which is what a Pascal programmer expects, and the collection property is right too. Only the shared name is wrong.

Recommended: make `Length` of a collection refuse rather than stringify — `Length` expects text; use `.Length` for a collection. A program that means the count says so, and one that means the rendering can write `Length(Str(L))`, which is what it is getting today by accident.

D-16 — `Val` always yields a Double, and `Max` never accepts one. (refers to [RT-009], [RT-010])

`Val('42')` is `42.0`, not `42`, so text that plainly holds an integer cannot be parsed into one — and the result then cannot be passed to `Max`, which refuses anything but Integers with `Max` expects Integers. The two built-ins are individually defensible and jointly unusable: `Max(Val(A), Val(B))` fails for every input.

Given [VAR-004], which refuses `var X : Integer := 1.5;` and even `var X : Double := 1;`, a program has no smooth path from parsed text to an Integer at all.

Recommended: let `Val` answer an Integer where the text has no point and a Double where it has one, matching how the literal rules already read the same characters [LEX-015], [LEX-020]; and let `Max` take numbers, promoting as every other numeric operator does [EXP-005]. Either change alone helps; both together close the gap.

D-17 — A type error says only "Type mismatch!" (refers to [ERR-006])

Every mismatch the checker finds produces the same five words, with no file, no line, no token and neither of the types involved. In a seven-line program the diagnostic is:

```
Uncaught: Type mismatch!
```

and nothing else. The parser and resolver, by contrast, print the file, the line, the source text and a caret under the offending token [ERR-005] — so the machinery for a good diagnostic already exists, is already used two phases earlier, and the checker simply does not reach for it.

⚠ This compounds with [ERR-002]: because a type error stops the program before any statement runs, a programmer gets no output to orient by either. The message is the only information available, and it carries none.

⚠ And with [FUN-006]: a parameter's declared type is unenforced, so mismatches surface in fewer places than a reader expects — which makes the ones that do surface harder to locate, not easier.

Recommended: raise through `Console.Error` with the offending token, as the parser does, and name both types — `Expected Integer, found String`. The token is in hand at every one of the five sites that raise this; the message is discarded rather than absent. This is the single cheapest improvement to the language's usability in this annex.

D-18 — `AssertTrue` reports a comparison it did not make. (*refers to [TST-012]*)

`AssertTrue(False)` fails with:

```
Assertion 'left = right' failed.
```

There is no left and no right. The message belongs to `AssertEqual`, which appends `Expected 'E' but got 'A'.` to the same stem, and `AssertTrue` reuses the stem and supplies no operands — so the reader is told an equality failed when a truth test did, and given nothing about what was actually false.

`Fail(M)` produces `Failed. M`, with two spaces, which is a third shape again.

Recommended: give `AssertTrue` its own stem — `Assertion failed: expected true.` — and keep the operand clause for the assertion that has operands. The three messages are read far more often than any other output this language produces, since a failing test is the one moment a programmer is looking.

⚠ Any change here alters the report, which [TST-008] specifies and which both processors must reproduce byte for byte — so it is a change to the observable surface and to `bootstrap/algol.c` in the same breath, not a cosmetic edit.

Annex E — what could be written in Algol-24 itself (*non-normative*)

The collections and the built-in functions are native today. This annex asks, for each, whether it is native because it *must* be or only because it always has been — and what one feature would have to be added to unbind it.

The question matters because everything moved out of the runtime is one less thing the C back end and the interpreter can disagree about, and one more thing a reader can look up in Algol-24 rather than in C.

⚠ Three rules do most of the pinning, and they are worth naming once rather than repeating: a class cannot be subscripted [TYP-010], cannot be iterated [TYP-011], and cannot expose a computed property [TYP-012]. A collection written in Algol-24 would work, but it would be written `L.Get(0)`, `L.Length()` and a loop over an index — a second-class citizen beside the built-ins.

E.1 The collections

Array — pinned, and rightly so.

`Array` is fixed-size, indexed in constant time, and holds arbitrary values. Nothing in the language can express that. `Buffer` stores bytes rather than values; a linked structure of class instances gives $O(n)$ access; and a class cannot be subscripted in any case. It is the one collection that is genuinely primitive.

Recommendation: keep it native, and treat it as the primitive the others are built on rather than as one collection among five.

Stack — unbound already, in all but syntax.

A `Stack` is `Push`, `Pop`, `Peek` and a count over a sequence. It can be written in Algol-24 today, in full, as a class holding a `List` — and has been: `spec/probes/TYP-012-stack-and-set-in-algol24.a24` is a working one, beside a `Set` whose membership test is a hand-written scan using no native `Contains`, and a `Mod` built from arithmetic alone. All three behave as the built-ins do.

Nothing about it is primitive: no literal claims its name, no subscript is needed, and its whole surface is method calls.

The only visible difference would be `S.Length()` against the built-in `S.Length` — [TYP-012] alone.

Recommendation: the best first candidate to move into a unit. It would prove the path with the least at stake, and the only thing it waits on is a getter.

Set — writable today; would want hashing later.

`Set` needs membership. Over a `List` with a linear scan it can be written immediately, and the probe named above does exactly that. A hashed version is also expressible: `Ord` yields a character's code point as an `Integer` — `Ord('A')` is 65 — so a string hash can be computed in the language without reaching for anything native.

Nothing pins it syntactically. `Set(L)` is an ordinary constructor call, not a literal form.

Recommendation: movable, and the performance question is separable — ship the linear version as a unit, hash it later if it matters.

List — pinned by its literal, not by its behaviour.

A growable sequence over `Array`, doubling when full, is straightforward Algol-24. What pins `List` is syntax: `[1, 2]` produces one, so the language itself hands the name out. Subscripting [TYP-010] and `for ... in` [TYP-011] would also have to be lifted, or every use would read `L.Get(I)`.

Recommendation: unbinding this one is worth doing only if user-defined subscript and iteration arrive first. Otherwise the unit is strictly worse to use than the built-in it replaces.

Map — pinned by its literal, doubly.

`[:]` and `[k:v]` produce a Map, so the name is claimed the way `List` 's is, and the natural `M[K]` needs [TYP-010] as well. The implementation itself is ordinary: two sequences, or `Array` buckets with an `Ord` - based hash.

⚠ Insertion order is specified behaviour, not an accident of the implementation, so any replacement must keep it.

Recommendation: same as `List` , and after it. The literal is the harder half: a language that lets a unit claim `[:]` is a much larger language.

E.2 The built-in functions

Native	Could it be written in Algol-24?
<code>Max</code> , <code>Mod</code>	Yes, today, from arithmetic alone.
<code>Copy</code> , <code>Pos</code>	Yes, from <code>Length</code> and subscripting a <code>String</code> , at a cost in speed.
<code>Val</code>	Yes, given <code>Ord</code> for digits.
<code>Ord</code> , <code>Char</code>	No. They convert between a character and its code point, and nothing else reaches that representation.
<code>Length</code>	No, for <code>String</code> . A user type's own length is [TYP-012].
<code>Str</code>	No. Rendering a <code>Double</code> in the specified shortest round-trip form needs the value's bits, which the language cannot see.
<code>clock</code> , <code>TextFile</code> , <code>FileExists</code> , <code>ParamCount</code> , <code>ParamStr</code>	No. The operating system is not otherwise reachable.
<code>Buffer</code>	No. It is a memory primitive with an explicit lifetime.
<code>Write</code> , <code>WriteLn</code>	No. Output is not otherwise reachable.

E.3 The one feature that unbinds the most

Of the three pins, **[TYP-012], the missing getter, is the cheapest and buys the most.** It is a declaration form rather than a semantic change; it makes `Stack` and `Set` writable as units indistinguishable from the built-ins; and it is needed by ordinary user classes regardless of whether any collection ever moves.

Subscript [TYP-010] is next, and iteration [TYP-011] after it — those two together are what `List` and `Map` wait on, and both are larger changes, because each means dispatching a language construct into user code.